AI-Assisted On-Call Queue: Budgeted Incident Detection for Cloud Microservices — Project Proposal (30 September draft)

(Alert Fatigue Reduction in Cloud Microservice Operations)

Author: Aishah Mohamed Alyammahi **Course:** Software Engineering and Programming **Date:** 30 September 2026 **Sector:** Cloud microservice operations (online platforms)

This is the Markdown draft of 30 September. The proposal as submitted on 5 October is `submission/Project_Proposal_200010549_Aishah_Alyammahi.pdf`. The five sections below are the proposal. Section 6 keeps the measured method and the GAIA numbers. The full course brief is answered in `FINAL_REPORT.md`.

1. Motivation

An on-call engineer for a cloud platform sees thousands of alerts a day and can check only a few. Most alerts are duplicates or several symptoms of one fault. The usual tools escalate by log severity and delete exact duplicates. They do not treat a sudden memory rise as one incident, and they do not stop when the engineer's attention runs out. Real faults are then missed because the queue is full, not because monitoring was silent. On the GAIA microservice data used here, most injected faults never write a log line at all, so a severity rule alone catches none of them.

Problem statement. On-call engineers lack a queue that stays inside a fixed daily budget and still surfaces confirmed memory faults ahead of log noise, so genuine incidents wait while duplicates are read.

2. Expected impact

A platform team can open one queue of at most K items for the day, see a memory jump as a single confirmed item, and leave paging off until they choose Live. Users of the platform benefit when a fault is noticed in about a minute instead of after the log noise has been cleared. The value depends on the budget being real, on shadow mode sending nothing, and on the evaluation using later days than the week the thresholds were learned from. This project does not replace the engineer: the queue ranks items, and a person decides what to act on.

3. Proposed approach

The working system is a Python service. FastAPI serves the queue, the API, and the connectors. SQLite holds alerts, jobs, and marks. Rules detect a memory jump (0.5 GB above the previous 15-minute median, two confirming points), a slow leak, and a monitoring gap. A cost-sensitive logistic regression ranks log alerts only, because a rule already finds the jump and most faults have no log line. Missing a fault costs ten times one extra item. The model is stored as a skops archive with a hash, a model card, a promotion gate, and rollback. Docker runs the app on the engineer's machine. Shadow mode is the default.

The four build milestones are done:

- **M1 — Data and baseline.** GAIA alerts, logs, and memory are aligned to UTC. The severity-plus-duplicates baseline and the Prometheus static and rate rules run on the same time split.
- **M2 — Alert classification.** The classifier is trained on the first week only. It does not score the rows it just trained on.

- **M3 — Memory jumps.** One jump is one incident. One-point blips are not shown. The jump is shown at its confirming point, not at the end of the day.
- **M4 — Budget.** The queue is first-come, at most K items a day, with log alerts capped at a quarter of K. K is reported at 5, 10, 20, and 50.

Human marks (Real or Noise) are collected in shadow mode and can retrain the classifier. They are not required for the GAIA result, which uses the injected-fault labels.

4. Deliverables and success criteria

Deliverables: this repository, setup in the root `README.md`, the design specification, automated tests, Docker, and `FINAL_REPORT.md`.

Success is a task someone can watch:

- An engineer opens the queue and sees at most K items for the day.
- A memory jump is one item, and it appears only after the second confirming point.
- Shadow mode sends nothing. Live paging happens only when the password, a channel, and the typed word Live are all set.
- On the later GAIA window, at K = 20, the jump detector beats the Prometheus rate rule, and the walk-forward weeks stay near that result. The numbers are in section 6 and are checked by `python run_validate.py`.

There were no human usability sessions. GAIA records every injected fault, which is the check used instead.

5. Timeline and feasibility

The core queue, the detectors, shadow mode, the tests, and the GAIA evaluation are built. What remains before the December presentation is the demo and the viva notes. The Word copies in `submission/` were regenerated from these Markdown files on 30 September 2026.

Out of scope, and acceptable to leave limited: automatic remediation, browser tests, the load test inside CI, a hosted cloud deployment, and an LLM summary of incidents. Sign-in sets a session cookie, a second password can read the queue but cannot change it, the audit log is also written to a file, and paging secrets are stored outside `ops.json`.

Synthetic or public data is enough for the submission. A pilot on a real team's alerts would need their permission and is not part of this semester.

6. Method and measured results

Data — GAIA

The project uses **GAIA (Generic AIOps Atlas)**, a free public dataset:

- **Big and real** — microservice logs and per-service container memory from about two months of operation (July–August 2021): 34,863 log alerts and 1,367 injected faults in the subset used here.
- **Trustworthy labels** — every fault was injected deliberately and recorded (a program holding extra memory for about ten minutes on one service), so the correct answer is known for each incident. Evaluation does not depend on the alerts' own severity fields.
- **Free to use** — github.com/CloudWise-OpenSource/GAIA-DataSet

The span supports a realistic time-based evaluation: learn from the first week (3,630 alerts, 144 faults), test on the following 55 days (31,233 alerts, 1,223 faults). The logs are in China time and the memory files in UTC, so both are aligned to UTC on loading.

Method (as built)

These are the checks each milestone had to meet. They are built.

M1 — Data pipeline and baseline

- Ingest GAIA alerts/logs/metrics; align alerts with the injected-fault records.
- Re-implement the baseline: fixed severity thresholds + exact duplicate suppression.
- Record baseline numbers (detection rate, reduction rate, MTTA) on the identical train/test split used everywhere else.

M2 — Alert classification

- Features from alert content and context (type, source, text similarity, recent history).
- Cost-sensitive training: missing a real fault is weighed far more heavily (10×) than raising one extra alert.
- *Done when:* the classifier beats baseline precision–recall at an equal review budget on the held-out test window.

M3 — Memory-jump episodes

- Mark a jump when memory rises more than 0.5 GB above the median of the previous 15 minutes on the same service, for at least two points (at most two minutes apart).
- The jump size is chosen on the training week only. A first version (training-week mean + 3 standard deviations) was replaced because a service whose normal level drifts is flagged for days.
- *Done when:* jumps overlap injected memory faults on the same service, and one-point blips are not reported.

M4 — Budget-aware decision policy

- One daily budget of K items, first come first served, with no look-ahead. A jump is shown at its confirming point if its rise by then passes a threshold learned on the training week; a monitoring gap after 10 minutes of silence; log alerts use at most 25% of K, and an alert inside a shown jump rides on it.
- Rank by expected cost so the top-K items fit the human budget; sweep K = 5, 10, 20, 50 per day.
- *Done when:* the queue beats a Prometheus-style rate rule at K=20 in real time; walk-forward mean stays near the one-shot test; weaker weeks and data gaps are reported, not hidden.

Stretch — human-in-the-loop

- Only uncertain alerts inside a memory jump are routed to a person, instead of every uncertain alert; report how decisions change when staff can handle more or fewer items.
- An on-call web app runs the queue in shadow mode (nothing is paged). People mark items Real or Noise, and those marks become labels for retraining on the company's own data.

Evaluation plan

Protocol

- Time-based split: train on the first week, test on the following 55 days — never random-shuffle.
- No leakage: patterns and thresholds are learned from the training window only.
- Budget sweep: results reported at K = 5, 10, 20, 50 reviews/day, not a single operating point.
- Baseline parity: the baseline runs on the identical split.

- Ground truth: GAIA's injected-fault records.

What this project is marked on

Claim	How it is shown
Beats the rules a team already runs, in real time	K=20: jumps + learned threshold 72.6% and full queue 70.2%, vs Prometheus-style rate rule 68.9%, static rule 57.2%, log severity 0%; fewer false items (1.0–1.6 vs 2.1 a day)
Not a lucky later month	Walk-forward (everything re-learned each week): jumps + threshold 74.8%, beats the rate rule 8/8 weeks ($p = 0.0039$); full queue 72.5% (95% CI 65.7–78.3%)
Jumps, not alert text, find the faults	71.0% of test faults have a jump and no log line
Near the ceiling	Best possible at K=20 is 84.9% (22 faults a day); the queue reaches 84–86% of it
Limits are stated	Every fault is 1 GB for 600 s; 10.8% of test faults fall in monitoring gaps (no data), shown as gap items; fold 7 is a 2.5-day metric outage

The 95% recall / 95% detection figures were an **early hope**. They are not the success test: only **17.9%** of test faults leave any log, and at K=20 no method can exceed 84.9%. Full argument: `FINAL_REPORT.md`.

Time to confirm is from the injection timestamp to the jump's confirming point: median **0.72 minutes**.

Results (test period, 1,223 faults)

Measure	Result
Jump overlap, no budget	86.1%; 99.6% of faults that have metric data
First version (mean + 3σ), for comparison	54.2%
Chance overlap (jumps shifted 1–6 h)	2.6%
Log severity baseline at K=20	0% — the lines inside a fault are warnings, not errors
Rate rule `delta(mem[5m]) > 0.5 GB` at K=20	68.9%, 2.1 false items a day
Full queue at K=20	70.2%, about 18 items and 1.6 false items a day
Why faults are missed at K=20	46% metric outage, 52% budget full, 2% detector
Other fault shapes (synthetic, real series)	0.5–2 GB steps ≥ 98%; leaks 73–99% with the leak rule; 0.25 GB not detected
Full queue at 5 / 10 / 50 items per day	19.5% / 38.6% / 85.8%
Walk-forward (K=20)	jumps + threshold 74.8%; full queue 72.5%; rate rule 70.7%

Cautions. Every GAIA fault is the same memory program, so this data favours a memory rule; on a real platform some faults will not touch memory, and the alert share of the budget matters more. Keep this dataset for the submission: the finding *is* that most injected faults leave no log.

7. Risks and mitigations

Risk	Mitigation
Memory series missing or flat	Keep the classifier path; the jump rule needs only the previous 15 minutes, not a training week
Results depend on GAIA faults all being memory faults	Report the alert-only numbers separately; retrain on a company's own labels collected in shadow mode
Detection rate never beats baseline at any budget	Revisit features and labels before adding model complexity
Marker still grades the old 95% table	Proposal and report now state the measured claim; `run_validate.py` checks the csv files
Changing to a second public dataset late	Do not. All 1,367 injections here are memory programs; extra metric types are not in the local files

8. Tools and resources

- Python (pandas, scikit-learn), Jupyter for analysis
- GAIA dataset (github.com/CloudWise-OpenSource/GAIA-DataSet)
- GAIA memory metric csv files (timestamp, value), one series per service
- Git for version control; all experiments reproducible from scripts

9. Expected outcome

A working, evaluated prototype that, at a fixed daily budget, catches more injected memory faults than severity + exact-duplicate grouping, with a walk-forward interval and a stated ceiling (faults that never move memory and never write a log). Alongside it, a local on-call web app that runs the same queue in shadow mode, so a platform team can collect its own labels before trusting it.

Beyond microservices. The same queue can be evaluated and tuned on any timestamped metric — orders, sensors, ad clicks, notification volume — from an **Evaluate my data** page, with no code. It is tested on a synthetic online shop and on five domains of the public NAB benchmark, and always reported against a random-chance line.